

Differentiable NAS

Differentiable NAS

1. DARTS paper (Liu et al., 2019) → arxiv.org/abs/1806.09055
2. DARTS official code (GitHub: [quark0/darts](https://github.com/quark0/darts))

Given a DAG, find the best assignment of (predefined) operations to edges (1 edge → 1 operation)

FIXED (predefined): LEARNED (by DARTS):

- Number of nodes • α values per edge (which op wins)
- Which edges exist • Network weights w (inside each op)
- Candidate op set
- Number of cells
- How cells stack

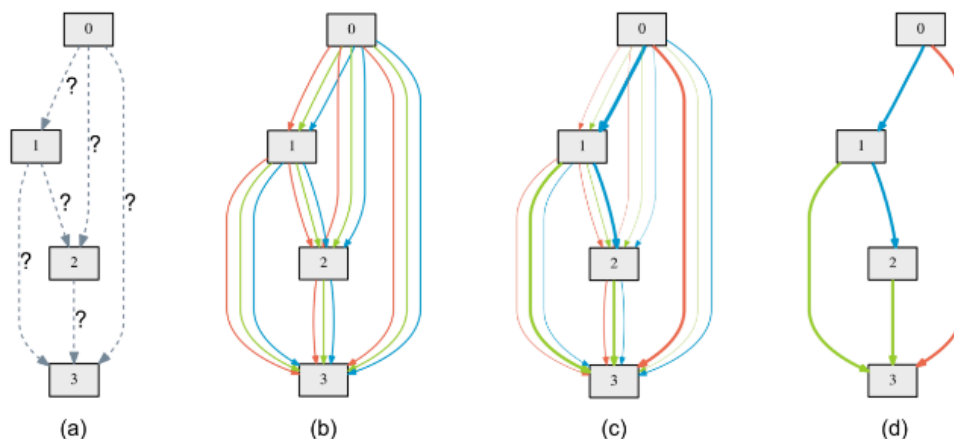


Figure 1: An overview of DARTS: (a) Operations on the edges are initially unknown. (b) Continuous relaxation of the search space by placing a mixture of candidate operations on each edge. (c) Joint optimization of the mixing probabilities and the network weights by solving a bilevel optimization problem. (d) Inducing the final architecture from the learned mixing probabilities.

A special *zero* operation is also included to indicate a lack of connection between two nodes. The task of learning the cell therefore reduces to learning the operations on its edges.

Training - how to choose a value over categorical building blocks?

In short -

1. Randomly initialize weight vector α_o
2. Apply softmax on α_o
3. Scale outputs of each architecture by its associated weight → a weighted sum of all outputs
4. Backprop with respect to α and w . The latter contains the weights of architectures, e.g. conv 3x3.

5. After training - $\arg\max$ over α at each edge and the architecture is ready.

2.2 CONTINUOUS RELAXATION AND OPTIMIZATION

Let $\mathcal{O}$ be a set of candidate operations (e.g., convolution, max pooling, *zero*) where each operation represents some function $o(\cdot)$ to be applied to $x^{(i)}$. To make the search space continuous, we relax the categorical choice of a particular operation to a softmax over all possible operations:

$$\bar{o}^{(i,j)}(x) = \sum_{o \in \mathcal{O}} \frac{\exp(\alpha_o^{(i,j)})}{\sum_{o' \in \mathcal{O}} \exp(\alpha_{o'}^{(i,j)})} o(x) \quad (2)$$

where the operation mixing weights for a pair of nodes (i, j) are parameterized by a vector $\alpha^{(i,j)}$ of dimension $|\mathcal{O}|$. The task of architecture search then reduces to learning a set of continuous variables $\alpha = \{\alpha^{(i,j)}\}$, as illustrated in Fig. 1. At the end of search, a discrete architecture can be obtained by replacing each mixed operation $\bar{o}^{(i,j)}$ with the most likely operation, i.e., $o^{(i,j)} = \arg\max_{o \in \mathcal{O}} \alpha_o^{(i,j)}$. In the following, we refer to α as the (encoding of the) architecture.

After relaxation, our goal is to jointly learn the architecture α and the weights w within all the mixed operations (e.g. weights of the convolution filters). Analogous to architecture search using RL (Zoph & Le, 2017; Zoph et al., 2018; Pham et al., 2018b) or evolution (Liu et al., 2018b; Real et al., 2018) where the validation set performance is treated as the reward or fitness, DARTS aims to optimize the validation loss, but using gradient descent.

Denote by $\mathcal{L}_{train}$ and $\mathcal{L}_{val}$ the training and the validation loss, respectively. Both losses are determined not only by the architecture α , but also the weights w in the network. The goal for architecture search is to find α^* that minimizes the validation loss $\mathcal{L}_{val}(w^*, \alpha^*)$, where the weights w^* associated with the architecture are obtained by minimizing the training loss $w^* = \arg\min_w \mathcal{L}_{train}(w, \alpha^*)$.

This implies a bilevel optimization problem (Anandalingam & Friesz, 1992; Colson et al., 2007) with α as the upper-level variable and w as the lower-level variable:

$$\min_{\alpha} \mathcal{L}_{val}(w^*(\alpha), \alpha) \quad (3)$$

$$\text{s.t. } w^*(\alpha) = \arg\min_w \mathcal{L}_{train}(w, \alpha) \quad (4)$$

Interesting subject - Zero-Cost NAS

ZERO-COST PROXIES FOR LIGHTWEIGHT NAS <https://arxiv.org/pdf/2101.08134>

Of course, the aforementioned approach requires a lot of compute. This work seems to be the remedy for that.

Idea:

"Look at the network at random initialization,
measure something cheap,
and predict which architecture will be best after training"
→ requires one forward/backward pass

But **what do we measure?**

- NASWOT — Activation Diversity: a good network maps different inputs to **different activation patterns**. If two inputs produce identical ReLU firing patterns, the network can't distinguish them — that's bad.
- Synflow — Gradient Flow: measures how well **gradients flow through the network**. Architectures where gradient signal dies out (vanishing gradients at init) will be hard to train.

- GradNorm: stronger gradients at init $\rightarrow$ easier to train $\rightarrow$ better final accuracy.